

Name: _____

Homework #01: Numbering Systems

1. Convert the binary numbers (unsigned) to decimal:
1) 0b11010100; 2) 0b01011010; 3) 0b00111100; 4) 0b10101011;

2. Convert the decimal numbers to binary:
1) 23; 2) 127; 3) 37; 4) 106.

3. Convert the hexadecimal numbers (unsigned) to decimal:
1) 0xE2; 2) 0x9A; 3) 0x3722; 4) 0xA282.

4. Convert the decimal numbers to hexadecimal:
1) 4096; 2) 832; 3) 1927; 4) 321.

5. Convert hexadecimal to binary and convert binary to hexadecimal
1) 0b00111100; 2) 0b10100011; 3) 0x9A; 4) 0x3722.

6. Convert the numbers into the specified number system (If you need a digit out of the range of 0-9 and A-F, circle the number to create a digit. For example, you can use $\textcircled{21}$ to represent the number 21.):
- 1) Convert $(1023)_5$ to base-8 (Radix = 8);
 - 2) Convert $(762)_8$ to base-5;
 - 3) Convert $(5\textcircled{18})_{20}$ to base-60;
 - 4) Convert $(A1)_{12}$ to base-20.
7. Change the negative decimal numbers into signed 8-bit binary, using all three representations including sign-magnitude, ones' complement, and two's complement:
- 1) 0; 2) -1; 3) -79; 4) -63
8. To add two 8-bit signed binary numbers using two's complement, simply align the values on the least significant bit and add, propagating any carry to the bit one position high. If you have a carry past the end of the word (overflow), just drop it. Convert the numbers in the following into signed binary using two's complement, add them, and convert it back to decimal. Check if the outputs are correct, if not, explain why.

- 1) $-23+38$; 2) $23+(-38)$; 3) $67+82$; 4) $(-63)+(-72)$

9. (Extra 10 pts, use additional sheets) Using any language that you prefer, e.g. C/C++, Python, Matlab, Java, etc., to develop a function which can convert your people soft number into a positioned number system base from 2 to 20. Use A to J to represent the digit 10 to 19. The radix can be defined as a parameter. You can either past your code or describe your algorithm here. Then show me 3 examples of the inputs and outputs of your function.
10. (Extra 10 pts, use additional sheets) If the processing unit of the processor has M positions (which is called bits in common computers), and each position has N states (radices), it can represent N^M (N to the power of M) states. That means the processor can represent the number from 0 to N^M . Let's assume the cost to build the processor is proportional to the number of the states, which are $M*N*C_0$, where C_0 is the cost of a state; For example, an 8-positioned base-2 unit can represent the numbers from 0 to 255 (2^8) with a cost of $16C_0$ ($2*8$); a 5-positioned base-3 unit can represent the numbers from 0 to 242 (3^5) with a cost of $15C_0$ ($3*5$). The ratios of the representable numbers and the cost are $(256/16C_0)$ and $(243/15C_0)$, respectively. Please consider:
- 1) If we want to have a processor capable of handling the number from 0 to 99,999. How many positions does the processing unit need if the unit is based on the radices of 2, 3, 5 or 8?
 - 2) Compare the ratio of the total numbers and costs for each case in 1).
 - 3) If the number goes to $[0, \infty)$, which base has the highest number to cost ratio?